AcademiGen

AI-Powered Academic Record Generator

Product Requirements Document | Version 1.0

Confidential • Full-Stack AI Web Application • April 2026

Automatically generate structured lab records, code documents, and academic reports from natural language.

Table of Contents

01 Product Vision

VISION	AcademiGen transforms the tedious process of creating academic records into a seamless, AI-driven experience — letting students focus on learning, not formatting.

1.1 Problem Statement

Every semester, thousands of engineering and science students spend hours — sometimes days — manually formatting lab records, program documentation, and academic reports. The problem is not the content; it is the repetitive, error-prone, time-consuming structure.

The Current Pain	The Cost
Manual formatting of each program	2–4 hours per record
Inconsistent structure across batches	Marks deducted for format errors
Copying boilerplate repeatedly	High cognitive load, low productivity
No standard template enforcement	Different outputs per student
Lost files, no version history	Work lost before submission

1.2 Why This Product Exists

AcademiGen is built on a simple premise: students should spend time understanding programs, not formatting documents. By automating structure, code formatting, output generation, and export, we eliminate busywork and standardize academic output.

Real-World Relevance

- Engineering colleges in India alone have 3.5M+ students submitting lab records annually
- Average student spends 6–10 hours/semester purely on formatting
- Teachers waste time correcting format errors instead of evaluating understanding
- Remote/hybrid learning created demand for digital, shareable, consistent records
- Institutions adopting digital submissions need automated, uniform document generation

1.3 Competitive Advantage

Feature	AcademiGen	Existing Tools (Word/LaTeX/Notion)
Template replication from upload	✅ Auto-parsed	❌ Manual setup
Code execution + output capture	✅ Built-in sandbox	❌ Not available
AI natural language input	✅ Full chatbot interface	❌ None
Academic-specific formatting	✅ Pre-built templates	Generic only
One-click PDF/DOCX export	✅ Instant	Manual formatting
Multi-language code support	✅ C, Python, Java, SQL	❌ None
Screenshot of output	✅ Auto-generated	❌ Manual screenshot

02 Target Users & Pain Points

AcademiGen serves a layered user ecosystem — each with distinct needs, workflows, and pain points.

2.1 Primary User — The Student

WHO	Undergraduate/postgraduate students in Computer Science, IT, Electronics, or any lab-intensive discipline who submit regular program records.

Student Pain Points

- Spending more time formatting than understanding the program logic
- Not knowing the exact format expected by their institution or teacher
- Repeated copy-paste of program headers, aim, algorithm, output every week
- Manual screenshotting of terminal output and inserting into Word documents
- Losing work due to no cloud sync or version history
- Struggling with LaTeX for technical documents
- Inconsistency between classmates' submissions causing confusion

2.2 Secondary User — The Teacher / Faculty

WHO	Faculty who define the lab record format, assign programs, review submissions, and grade based on document quality and accuracy.

Teacher Pain Points

- Reviewing inconsistent student submissions with different formats
- No standard enforcement mechanism for digital record submissions
- Time wasted providing formatting feedback instead of content feedback
- Difficulty verifying code output authenticity
- No easy way to distribute a standard template to all students

2.3 Tertiary User — The Institution / Administrator

Institutional Pain Points

- No centralized record of all student lab submissions
- Lack of uniformity across different batches and sections
- Difficulty integrating with existing LMS (Moodle, Google Classroom, etc.)
- No analytics on which programs students struggle with most
- Compliance with accreditation standards (NBA, NAAC) requires standardized documentation

03 Core Features — Detailed Specification

3.1 AI Chatbot Interface

The primary entry point for all users. Instead of filling forms, students describe what they want in plain English (or their regional language).

Capability	Description
Natural language input	"Generate a Python record for bubble sort with algorithm and output"
Context awareness	Remembers previous messages in session to refine output iteratively
Multi-turn dialogue	Ask follow-up questions: "Add a flowchart section" or "Change the font"
Guided prompting	Suggests missing fields: "Do you want to add the algorithm?"
Voice input support	Speak instructions using Web Speech API or Whisper integration
Language support	English + regional input (Hindi, Telugu, Tamil) via translation layer

3.2 Template Upload & Parsing

Students can upload a sample PDF or Word document. The system parses the structure and replicates it.

Template Parsing Capabilities

- PDF parsing: Extract layout, fonts, column structure, margins, header/footer zones
- DOCX parsing: Read styles, heading hierarchy, table structures, list types
- Header detection: Logo position, college name, batch info, date fields
- Footer detection: Page numbers, subject name, signature blocks
- Automatic style extraction: Font family, size, weight, color, spacing
- Section detection: Identify 'Aim', 'Algorithm', 'Code', 'Output' zones
- Multi-template library: Pre-built templates for DS, Python Lab, DBMS, Networks, OS
- Template preview before application

- Partial template use: Apply only header/footer from template, generate rest with AI

3.3 Code Execution & Output Generation

The most powerful feature — students paste or describe code, and the system executes it in a safe sandbox and captures the output.

Language	Execution Engine	Output Types
Python	Pyodide (browser) / Judge0 API	Text, graphs (matplotlib), tables
C / C++	Judge0 API / GCC sandbox	Terminal text output
Java	Judge0 API / OpenJDK sandbox	Console output
SQL	SQLite WASM / PostgreSQL container	Table results
JavaScript	Browser sandbox (iframe)	DOM, console output
Shell/Bash	Docker container (restricted)	Command output

SAFETY All code runs in isolated Docker containers or WASM sandboxes with no network access, file system restrictions, memory limits (256MB), and time limits (10s). No code runs on the main server.

3.4 Output Screenshot Generation

Automatically generate visual representations of terminal/program output, formatted to match handwritten or digital record aesthetics.

Screenshot Generation Pipeline

- Execute code → capture stdout/stderr as raw text
- Render text in a styled terminal emulator (dark theme, monospace font)
- Generate PNG/JPEG screenshot using html2canvas or Puppeteer
- Insert screenshot into document at the designated 'Output' section
- Option: Show both raw text output AND screenshot in document
- For graphical programs (matplotlib plots), capture the rendered image directly
- Customizable output box styling (border, background, font size)

3.5 Document Editor

A full-featured editor that combines the simplicity of a word processor with developer-grade tools.

Feature	Details
Rich text editing	Bold, italic, underline, font change, color picker
Code editor block	Monaco Editor (VS Code engine) with syntax highlighting
Drag-and-drop sections	Reorder Aim, Algorithm, Code, Output blocks freely
Live preview pane	See formatted document in real-time as you type
Image insertion	Paste, upload, or auto-insert output screenshots
Table support	Insert and style tables for algorithm/truth tables
Section collapse	Collapse/expand sections for focused editing
Undo/Redo stack	Full history with Ctrl+Z/Ctrl+Y support
Auto-save	Every 30 seconds to local storage and cloud

3.6 Export Options

Export Capabilities

- PDF export: High-fidelity, print-ready, preserves all formatting
- DOCX export: Editable Word document, compatible with MS Office and Google Docs
- Image export: Export each page as PNG/JPEG (useful for WhatsApp submission)
- Bulk export: Export all programs in a batch as a single merged PDF
- Email share: Send generated document directly to teacher's email
- Cloud storage: Save to Google Drive or OneDrive with one click
- QR code: Generate a QR code linking to the document's cloud version

3.7 Version History & Collaboration

Version & Collaboration Features

- Automatic version snapshots: Every save creates a recoverable version
- Named versions: 'Draft 1', 'After teacher review', 'Final submission'
- Diff view: See what changed between versions (highlighted changes)
- Restore any version with one click
- Shareable link: Generate a read-only link for teacher to review

- Comment system: Teacher adds inline comments on the shared document
- Real-time collaboration (future): Multiple students work on one record

04 AI Features — Advanced Intelligence Layer

AI CORE

AcademiGen's AI layer is not just a chatbot add-on — it is the central intelligence that understands academic context, generates code, validates output, and guides the student from raw idea to polished document.

4.1 Natural Language → Structured Record Conversion

The single most powerful workflow: a student types one sentence and gets a complete, formatted record.

EXAMPLE

Input: 'Python program to implement stack using linked list with all operations' → Output: Complete record with Aim, Algorithm (step-by-step), Code (working, formatted), Output (executed), Result.

NL-to-Record Capabilities

- Parse program name, language, data structure/algorithm type from free text
- Identify required sections based on subject context (DS lab ≠ Python lab)
- Auto-generate Aim: 'To implement stack using linked list in Python'
- Auto-generate Algorithm: Numbered, step-by-step pseudocode/description
- Auto-generate working code in the specified language
- Execute generated code and capture actual output
- Generate Result: 'The program was successfully executed...'
- Detect subject context: 'DBMS experiment' triggers SQL-specific template

4.2 AI Code Intelligence

AI Code Feature	Description
Auto-complete code	Start typing and AI completes the function based on context
Error detection & fix	Identify syntax/logic errors and suggest corrections
Code explanation	Click 'Explain' on any function to get plain-English description
Code optimization	Suggest more efficient alternatives with complexity

	analysis
Missing code generation	Detect if code is incomplete and generate the missing parts
Language translation	Convert Python code to C or Java automatically
Comment generation	Add inline comments to all functions automatically
Complexity analysis	Display time/space complexity (O notation) for code blocks

4.3 Smart Template Intelligence

Template AI Features

- Upload any sample PDF: AI extracts layout rules and applies them to new documents
- Font detection: Identify fonts used in uploaded template (OCR + PDF metadata)
- Spacing inference: Learn paragraph spacing, line height, margin rules from sample
- Section auto-labeling: Detect 'Aim', 'Program', 'Output' zones without user marking
- Header/footer cloning: Exact replication including logo placement and text fields
- Style transfer: Apply visual style of uploaded document to any new content
- Template suggestions: Based on program type, suggest the best matching template

4.4 Voice Input Interface

Students can dictate their requirements instead of typing — especially useful on mobile or when hands are busy.

Voice Input Features

- Browser-native Web Speech API integration for real-time transcription
- Whisper API integration for higher accuracy (server-side fallback)
- Regional language support: Transcribe Hindi, Telugu, Tamil descriptions
- Voice commands: 'Next section', 'Add code', 'Generate output', 'Export PDF'
- Dictation mode: Long-form voice to full document section
- Punctuation inference: AI adds commas, periods, paragraphs from natural speech

4.5 'Explain This Program' Feature

USE	Student submits a C program for a viva. They click 'Explain' and get a teacher-

CASE	ready explanation of what the program does, line by line and concept by concept.

Explanation Features

- Line-by-line explanation in plain English
- Concept-level summary: 'This program uses a recursive approach to traverse a binary tree'
- Viva Q&A generation: 5–10 likely viva questions with answers based on the program
- Flowchart generation: Auto-generate a flowchart from the code logic
- Complexity explanation: 'This loop runs n^2 times because...'
- Beginner/Advanced mode: Toggle explanation depth

4.6 AI Quality & Suggestions Engine

AI Quality Features

- Grammar check on all text sections (Aim, Algorithm, Result)
- Academic tone enforcer: Rewrite casual text in formal academic language
- Completeness check: Alert if required sections are missing
- Output validation: Verify that code output matches what's documented
- Plagiarism risk check: Flag if generated content closely matches known sources
- Improvement suggestions: 'Your algorithm section could be more detailed'
- Format consistency check: Ensure consistent font/spacing throughout

05 UI/UX Design Specification

5.1 Application Page Structure

Page / Screen	Purpose & Key Elements
Landing / Home	Value proposition, 'Start a New Record' CTA, template gallery preview, recent documents
Chat Interface	AI chatbot panel, message history, quick action chips, voice input button
Editor	Left: Section navigator | Center: Rich editor | Right: Live preview panel
Preview	Full-page document preview with page layout, zoom controls, annotation mode
My Records	Grid/list of all saved records, search, filter by subject/date, bulk export
Export & Share	Format selection, quality settings, share link, email, cloud upload
Settings	Template preferences, default language, font settings, account info
Teacher Portal	Upload official template, view submissions, add comments, export gradebook

5.2 Editor Layout — Three-Panel Design

The editor is the heart of the product. It uses a three-panel layout optimized for document creation:

Left Panel	Center Panel	Right Panel
Section navigator	Active text editor	Live document preview
Drag-to-reorder sections	Code editor (Monaco)	Updates on every keystroke
Add/remove section buttons	AI suggestions inline	Zoom in/out controls
Progress indicator	Formatting toolbar	Page border simulation
Template switcher	Output display area	Scroll synchronized with editor

5.3 Student-Friendly Design Principles

Design Principles

- Zero learning curve: First-time users complete a record in under 10 minutes
- Mobile-first responsive: Works on phones (common device for Indian students)
- Dark mode support: Reduces eye strain during night study sessions
- Progress indicators: 'Your record is 80% complete — add the output section'
- Onboarding tour: Interactive walkthrough on first login
- Micro-animations: Smooth transitions that feel modern and responsive
- Accessibility: Screen reader support, keyboard navigation, high-contrast mode
- Offline mode: Continue editing without internet; sync when back online

06 System Architecture

ARCHITECTURE	AcademiGen follows a modern, scalable microservices architecture with a React frontend, Node.js/Python backend, isolated code execution containers, and AI API integrations.

6.1 Frontend Architecture

Component	Technology & Role
UI Framework	React 18 + Vite — fast development, component-based UI
State Management	Zustand — lightweight, scales with document state
Code Editor	Monaco Editor (VS Code engine) — syntax highlighting, IntelliSense
Rich Text Editor	TipTap (ProseMirror-based) — extensible, custom nodes for code blocks
PDF Preview	PDF.js — render PDF preview in browser without plugins
File Processing	pdf-lib, mammoth.js — client-side PDF/DOCX parsing
Code Execution (Python)	Pyodide — run Python in browser via WebAssembly
Voice Input	Web Speech API + Whisper.js fallback
Styling	Tailwind CSS + shadcn/ui components
Routing	React Router v6

6.2 Backend Architecture

Service	Technology & Role
API Server	Node.js + Express.js — REST API, file handling, auth
AI Orchestration	Python FastAPI — handles Anthropic/OpenAI API calls, prompt management
Template Parser	Python (pdfplumber, python-docx) — extract structure from uploads
Code Execution Service	Judge0 open-source instance — REST API for C, Java, SQL execution
Document Generator	Node.js + docx library + PDFKit — generate final DOCX/PDF

File Storage	AWS S3 / Supabase Storage — store templates, outputs, user files
Authentication	Supabase Auth / Clerk — JWT-based, OAuth (Google)
Database	PostgreSQL (via Supabase) — user data, records, version history
Cache	Redis — cache generated documents, execution results
Queue	BullMQ (Redis) — async document generation jobs

6.3 AI Integration Layer

AI Component	Service & Purpose
Primary LLM	Anthropic Claude API — code generation, explanation, NL-to-record
Secondary LLM	OpenAI GPT-4o — fallback, voice transcription, grammar check
Code Execution AI	Code Llama (self-hosted option) — offline code suggestions
Document Understanding	Google Document AI — parse complex PDF layouts
Image Generation	Canvas API + html2canvas — terminal screenshot generation
Voice	OpenAI Whisper API — speech-to-text for voice input
Translation	DeepL API / LibreTranslate — regional language support

6.4 Code Execution Architecture

Safe, isolated code execution is critical. The system uses a layered approach:

Layer 1 — Browser	Python via Pyodide (WebAssembly). No server required. Zero network risk. Handles 80% of student use cases.
Layer 2 — Judge0 API	C, Java, SQL, and other languages. Open-source, self-hostable. Each execution in isolated container with 256MB RAM, 10s timeout, no network.
Layer 3 — Docker Sandbox	Institution-hosted Docker containers for advanced use. Nsjail security, read-only filesystem, CPU limits. Full isolation.

07 Tech Stack — Free & Scalable

7.1 Complete Technology Stack

Category	Technology	Cost
Frontend Framework	React 18 + Vite	Free / Open Source
UI Components	Tailwind CSS + shadcn/ui	Free / Open Source
Code Editor	Monaco Editor	Free / MIT License
Rich Text	TipTap (ProseMirror)	Free tier available
Backend API	Node.js + Express.js	Free / Open Source
AI Orchestration	Python FastAPI	Free / Open Source
Primary AI	Anthropic Claude API	Pay-per-use (very cheap)
Secondary AI	OpenAI GPT-4o mini	Pay-per-use (cheap)
Code Execution	Judge0 (self-hosted)	Free (open source)
Database	Supabase (PostgreSQL)	Free tier: 500MB
File Storage	Supabase Storage	Free tier: 1GB
Authentication	Supabase Auth	Free: 50,000 MAU
PDF Generation	PDFKit / pdf-lib	Free / Open Source
DOCX Generation	docx.js	Free / MIT
PDF Parsing	pdfplumber (Python)	Free / Open Source
DOCX Parsing	mammoth.js	Free / Open Source
Voice Input	Web Speech API	Free (browser-native)
Voice (fallback)	OpenAI Whisper API	Pay-per-use (cheap)
Hosting (Frontend)	Vercel / Netlify	Free tier available
Hosting (Backend)	Railway / Render	Free tier: 512MB RAM
Code Exec Hosting	Fly.io Docker	Free tier available
Cache	Upstash Redis	Free tier: 10,000 req/day

08 Security Considerations

8.1 Code Execution Security

Sandbox Security Measures

- Network isolation: Code containers have zero internet access (block all outbound)
- Filesystem: Read-only except /tmp, max 10MB disk write
- Memory limit: 256MB per execution, OOM-killed if exceeded
- CPU limit: Max 2 CPU cores, 10-second execution timeout
- Process isolation: Each execution in separate container, destroyed after completion
- No system calls: Seccomp profile blocks dangerous syscalls (fork, exec, socket)
- Code scanning: Pre-scan for import of os, sys, subprocess before execution
- Rate limiting: Max 20 executions per user per hour

8.2 File Upload Security

Upload Security Measures

- File type validation: Server-side MIME type check (not just extension)
- Size limits: Max 10MB per file upload
- Antivirus scan: ClamAV integration for uploaded files
- Sandboxed parsing: Parse uploaded PDFs/DOCX in isolated Python process
- No executable uploads: Block .exe, .sh, .py uploads in template section
- Content security: Strip macros from uploaded DOCX before processing
- Storage isolation: User files stored in private S3 buckets with signed URLs

8.3 API & Data Security

API & Data Security Measures

- All API keys stored server-side only — never exposed to client
- JWT authentication with 1-hour expiry + refresh token rotation
- HTTPS enforced everywhere (HSTS headers)
- Input sanitization: All user text sanitized before AI API calls

- Prompt injection prevention: User input wrapped in system-level guards
- CORS: Strict origin whitelist
- Rate limiting: 100 requests/hour per IP on public endpoints
- GDPR compliance: User data deletion on request, no third-party data sharing

09 Challenges & Limitations

Challenge	Mitigation Strategy
PDF layout parsing accuracy	Combine pdfplumber + Google Document AI; allow manual correction fallback
AI hallucinating code output	Always execute code; never use AI-generated output as ground truth
Code execution performance	Cache results; Pyodide for Python (browser); async Judge0 for others
Regional font support in PDF	Embed font files in PDF; use system fonts for common languages
Large file PDF parsing	Stream-parse, process page by page; enforce 10MB limit
Voice recognition accuracy	Whisper fallback for low-accuracy transcriptions; manual correction
Scaling code execution	Auto-scale Judge0 containers; queue with BullMQ; rate limit heavy users
Template diversity	Crowdsource templates; allow community submissions with review
Offline capability	IndexedDB for local storage; Pyodide for offline Python; sync on reconnect
AI cost at scale	Cache AI responses; use smaller models for simpler tasks; offer premium tiers

10 Future Enhancements

VISION 2030 AcademiGen evolves from a document generator into a complete academic intelligence platform — the 'Notion + GitHub Copilot' for students.

10.1 Near-Term Enhancements (6–12 months)

Near-Term Features

- Mobile app (React Native) — generate records on the go, voice-first interface
- Offline desktop app (Electron) — full functionality without internet
- LMS integrations — submit directly to Moodle, Google Classroom, Canvas
- Batch generation — upload a list of 20 program titles, generate all records at once
- Teacher dashboard — view all student submissions, add grades and comments

10.2 Long-Term & Visionary Features

Visionary Feature	Description
Auto Handwriting Simulation	Generate document in a handwritten style matching the student's own handwriting sample (OCR + font generation)
AR/VR Record Preview	View your lab record in 3D space, flip through pages using hand gestures in AR
Teacher Verification System	Blockchain-based timestamp and hash of each submission for tamper-proof records
Auto Grading Engine	AI grades each section (Aim, Algorithm, Code, Output) based on rubric and provides score + feedback
Peer Learning Network	Share approved records (anonymized) as learning resources; 'See how others solved this'
Code Visualization Mode	Auto-generate animated step-by-step visual of algorithm execution (sorting, tree traversal)
Smart Plagiarism Detection	Compare submission against class batch to detect copied records
Adaptive Template Learning	System learns your teacher's preferences from feedback and automatically adjusts formatting
Multi-Modal Input	Take a photo of a handwritten algorithm; AI converts it to

	typed text in the record
Curriculum Integration	Import syllabus PDF → auto-generate all program stubs for the entire semester at once

11 Development Roadmap

Phase 1
Months 1–3

MVP — Core Functionality

- ✓ Basic chat interface with Claude API integration
- ✓ Pre-built templates for Python Lab and Data Structures
- ✓ Manual code input with Monaco Editor
- ✓ Python execution via Pyodide (browser-based, no server needed)
- ✓ Terminal output display and screenshot capture
- ✓ PDF export using PDFKit
- ✓ User authentication via Supabase
- ✓ Cloud save to Supabase Storage
- ✓ 5 pre-built templates (Python, DS, C, DBMS, Networks)

Phase 2
Months 4–6

Full-Featured System

- ✓ Template upload & parsing (PDF + DOCX)
- ✓ Judge0 integration for C, Java, SQL execution
- ✓ DOCX export with full formatting preservation
- ✓ Version history and document management
- ✓ Teacher portal with shareable links and comment system
- ✓ Voice input via Web Speech API
- ✓ Drag-and-drop section reordering
- ✓ Mobile-responsive UI with dark mode
- ✓ Batch export of multiple records

Phase 3
Months 7–12

AI-Powered Intelligence

- ✓ Full NL-to-record pipeline (aim + algorithm + code + output in one prompt)
- ✓ AI code error detection and auto-fix
- ✓ 'Explain this program' and viva Q&A generation
- ✓ Smart template replication from uploaded PDFs
- ✓ Academic tone enforcer and grammar check
- ✓ Auto flowchart generation from code
- ✓ Regional language support (Hindi, Telugu, Tamil)
- ✓ LMS integrations (Google Classroom, Moodle)
- ✓ Mobile app (React Native)
- ✓ Institution white-label version

12 Unique & Creative Ideas — What Sets AcademiGen Apart

INNOVATION These 12 ideas are not found in any existing academic tool. They represent the creative vision that transforms AcademiGen from a utility into a platform.

12.1 The 12 Signature Innovations

# Innovation	Why It's Unique & Valuable
1. Output Authenticity Score	AI compares expected output from code logic vs actual executed output and gives an 'Authenticity Score' — helps teachers verify genuine submissions
2. Viva Prep Mode	After generating a record, enter 'Viva Mode' where the AI acts as an examiner and asks questions about the program — students practice answering
3. Handwriting Font Generator	Student uploads a sample of their handwriting → system creates a custom font → entire document appears handwritten (for institutions requiring handwritten records)
4. Algorithm Animator	Auto-generate an animated GIF or HTML5 animation showing step-by-step execution of the algorithm (e.g., watch bubble sort in action) — embed in digital record
5. Semester Record Bundle	At end of semester, generate a complete, indexed, print-ready PDF record book with cover page, index, and all programs — one click for entire semester
6. Smart Similarity Detector	When entire class uses the same tool, detect suspiciously similar records and flag them — gives teachers insight into copy-paste submissions
7. WhatsApp Bot Integration	Send program name via WhatsApp → receive formatted PDF in 30 seconds. Zero app installation needed. Ideal for resource-constrained students.
8. Teacher Template Marketplace	Teachers upload and monetize their institution's official templates. Students pay ₹29 to get the exact format their college uses. Revenue share for teachers.
9. AI Code Narrator	Generate an audio MP3 of someone 'narrating' the code explanation — like a podcast episode of your program. Students listen while commuting.
10. Multi-Language Code Mirror	Write once in Python, instantly see the equivalent C and Java code side by side. Great for understanding language differences for exams.
11. Record Health Dashboard	Visual analytics: 'You've submitted 12 records this

	semester, your average completeness is 87%, Algorithm section is consistently weak' — actionable insights.
12. Offline-First PWA	Full Progressive Web App that works 100% offline using IndexedDB + Pyodide. No internet? No problem. Sync everything when back online.

A Appendix — Quick Reference

A.1 Glossary

Term	Definition
Lab Record	Academic document submitted by students containing program details, code, and output
Template Replication	Parsing an uploaded document to extract and apply its visual structure to new content
Code Sandbox	Isolated execution environment where code runs safely without system access
Monaco Editor	The code editor that powers VS Code, embedded as a React component
Pyodide	Python runtime compiled to WebAssembly, runs Python in the browser without a server
Judge0	Open-source code execution API supporting 60+ programming languages
TipTap	Headless rich-text editor framework built on ProseMirror
Supabase	Open-source Firebase alternative — provides database, auth, and file storage

A.2 Document Information

Field	Value
Product Name	AcademiGen
Document Type	Product Requirements Document (PRD)
Version	1.0
Date	April 2026
Status	Draft — Ready for Engineering Review
Author	Product & Engineering Team
Next Review	May 2026
Target Launch	Q3 2026 (MVP)

AcademiGen — Turning Effort into Excellence, Automatically.

This document is confidential and intended for internal use only.